MARISOLLIFLIX

APLICACIÓN WEB PARA REGISTRAR, CALIFICAR Y RANKEAR PELÍCULAS, ANIMES Y SERIES GEEK

Documento de planeación Scrum · Frontend y Backend

ESTUDIANTES
María José Montepeque Zet
Allison Rocío Vargas Mejía

DOCENTE INVOLUCRADO
Eduin Salas

CAMPUSLANDS
RUTA: NODE.JS
GUATEMALA
2026

MarisolliFlix · Aplicación web para registrar, calificar y rankear películas

Situación problema

Quienes disfrutan de las películas, los animes y las series geek suelen llevar su opinión repartida entre notas del celular, mensajes de chat y listas sueltas en distintas plataformas. Con esa forma de trabajo es común que no se recuerde qué se vio, qué nota se le puso a cada título, qué se quiere ver después ni quién más lo recomienda. Además, las calificaciones de unas pocas personas pesan igual que las de muchas, y las opiniones viejas pesan igual que las recientes, por lo que no hay un orden confiable para decidir qué ver.

El problema crece cuando cualquiera puede agregar información sin control: títulos repetidos, películas sin revisar, reseñas que alguien modifica a nombre de otra persona o votos que se cuentan dos veces. Esto genera un catálogo desordenado y poca confianza en el ranking.

MarisolliFlix nace como una aplicación web con dos partes independientes. El backend es una API REST en Node.js y Express con base de datos MySQL que guarda el catálogo (112 títulos con sinopsis y portada), las categorías, las reseñas con calificación de 1 a 10, los likes y dislikes y la lista personal de cada usuario. El frontend, hecho en HTML, CSS y JavaScript puro, consume esa API y permite explorar el catálogo, buscar, ver el trailer y dónde ver cada título, reseñar, votar, guardar en “Mi lista” y proponer títulos nuevos que un administrador aprueba o rechaza. El ranking es ponderado: combina la calificación, los votos de cada reseña y su antigüedad, y las operaciones sobre reseñas y votos se ejecutan dentro de transacciones para que los datos nunca queden a medias.

Levantamiento de requerimientos

Los requerimientos se obtuvieron a partir del enunciado del proyecto entregado por Campuslands, que describe las funcionalidades mínimas, los requisitos de seguridad y los requisitos técnicos. Con base en ese análisis se siguió este proceso:

- Identificación de los actores del sistema: el Visitante, que explora el catálogo; el Usuario con cuenta, que reseña, vota, guarda películas y propone títulos; el Administrador, que modera el catálogo y las categorías; y el Trainer, que evalúa la entrega.
- Agrupación de las funcionalidades en seis módulos: autenticación y roles, categorías, películas, reseñas y votos, ranking y listado, y la interfaz web (frontend).
- Redacción de 15 historias de usuario con el formato “Yo como [actor], quiero poder… Tomando los datos: …”, cada una con criterios de aceptación y restricciones.
- Priorización de las historias (Alta, Normal y Baja) y distribución en tres sprints dentro del tablero de ClickUp.

- Identificación de las operaciones que requieren transacciones: crear, editar y eliminar reseñas y registrar o cambiar likes y dislikes.
- Decisión técnica documentada: el enunciado propone MongoDB, pero el equipo usa MySQL 8 con el driver oficial mysql2 (sin ORM), la base de datos trabajada en el curso. Las transacciones reales se resuelven con tablas InnoDB y el ranking ponderado con vistas SQL.

Requerimientos

Requerimientos funcionales

Tabla 1

Requerimientos funcionales

Código	Requerimiento	Descripción	Prioridad
RF01	Crear una cuenta	El visitante se registra con nombre de usuario, correo y contraseña de 8 a 72 caracteres (POST /auth/register). La contraseña se guarda con hash bcrypt y nunca se devuelve. El formulario avisa del error sin enviar nada.	Alta
RF02	Iniciar sesión	POST /auth/login devuelve un token JWT. El frontend lo guarda en localStorage y lo envía en el header Authorization: Bearer. El login permite 5 intentos fallidos cada 15 minutos por IP y correo.	Alta
RF03	Tener roles de usuario y administrador	El middleware authorize('admin') protege las rutas de administración. El administrador se crea con un seed a partir del .env y el frontend solo muestra sus opciones a quien lo es.	Alta
RF04	Gestionar las categorías	El administrador crea, edita y elimina categorías (nombre único). Se rechaza eliminar una categoría con películas (409). Todos pueden consultarlas para filtrar.	Alta
RF05	Registrar películas, series y animes	Cada título tiene título único (sin distinguir mayúsculas), descripción, categoría, año, tipo, portada, trailer de YouTube, plataformas e IMDb. El administrador edita y elimina.	Alta
RF06	Proponer y aprobar títulos	El usuario propone un título que queda pendiente; el administrador lo aprueba o lo rechaza (PATCH /movies/:id/status). Solo las aprobadas aparecen en el catálogo.	Alta
RF07	Escribir una reseña con calificación	El usuario escribe una reseña por película con título, comentario y calificación de 1 a 10. Se ejecuta dentro de una transacción.	Alta
RF08	Editar y eliminar mi reseña	Solo el autor edita o elimina su reseña (403 si no lo es); al eliminarla se eliminan sus votos y el ranking se recalcula.	Normal
RF09	Dar like o dislike a las reseñas	Un voto por usuario y reseña; repetir el mismo tipo quita el voto y el otro tipo lo cambia. No se puede votar la propia reseña.	Normal
RF10	Ver un ranking justo	El puntaje es ponderado: pesa la calificación, los likes y dislikes y la antigüedad de cada reseña, con promedio bayesiano y desempate total.	Alta
RF11	Explorar el catálogo	Listado con búsqueda por título, filtro por categoría, orden (ranking, popularidad, recientes, título, año o nota de IMDb) y paginación resueltos por el backend.	Alta
RF12	Ver el detalle de una película	Sinopsis, portada, trailer, “dónde ver”, nota de IMDb y reseñas con sus likes, en una ventana emergente y en	Normal

Código	Requerimiento	Descripción	Prioridad
		una página completa.	
RF13	Administrar desde un panel	Pestañas de categorías, películas y pendientes para el administrador, con confirmación antes de borrar.	Alta
RF14	Guardar películas en Mi lista	El usuario guarda y quita títulos de su lista personal (GET/POST/DELETE /watchlist).	Normal
RF15	Recibir errores claros y seguros	Errores centralizados con formato único, validación de entradas, límite de peticiones y consultas parametrizadas.	Alta
RF16	Consultar la documentación del API	Swagger (OpenAPI 3) en /api/v1/docs con ejemplos y respuestas de cada endpoint; versión 1.0.0 con semver.	Normal

Requerimientos no funcionales

Tabla 2

Requerimientos no funcionales

Código	Tipo	Descripción
RNF01	Tecnología	Backend en Node.js 20+ con Express 5; frontend en HTML, CSS y JavaScript puro con módulos ES, sin frameworks ni librerías de interfaz.
RNF02	Persistencia	Base de datos MySQL 8 con el driver oficial mysql2 (sin ORM), esquema normalizado hasta 4FN: 13 tablas y 2 vistas InnoDB.
RNF03	Consistencia	Las operaciones sobre reseñas y votos se ejecutan en transacciones (BEGIN, COMMIT y ROLLBACK) con la función conTransaccion.
RNF04	Diseño	Programación orientada a objetos: clases, herencia, clase abstracta, encapsulamiento con atributos privados e inyección de dependencias.
RNF05	Arquitectura	Estructura por capas: ruta, validación, controlador, servicio, repositorio y MySQL, con DTOs y entidades.
RNF06	Seguridad	JWT con passport-jwt, contraseñas con bcryptjs, CORS restringido, límite de 100 peticiones cada 3 minutos por IP, SQL parametrizado y CSP estricta en el sitio.
RNF07	Calidad de datos	Validación con express-validator en todas las entradas (tipos, largos, rangos, URLs, lista cerrada de plataformas).
RNF08	Usabilidad	Interfaz responsive (menú hamburguesa bajo 700 px), foco visible, soporte de teclado, prefers-reduced-motion y estados de carga, vacío y error.
RNF09	Rendimiento	Portadas en WebP con srcset (220 y 400 px) y JPG de respaldo; imagen para compartir de 101 KB.
RNF10	Documentación	README en ambos repositorios y Swagger con la referencia de la API.
RNF11	Control de versiones	Repositorios en GitHub con una rama por tarea, Pull Requests hacia dev usando una plantilla y commits con formato Conventional Commits.
RNF12	Verificación	150 pruebas automáticas con node:test (autenticación, películas, categorías, reseñas, ranking, seguridad, Swagger y clases POO).
RNF13	Despliegue	Contenedores Docker (MySQL, API y Caddy) detrás de Cloudflare, con despliegue automático al hacer push a main.

Historias de usuario con criterios de aceptación

HU-01 · Crear una cuenta

Código del requerimiento	RF01 (HU-01) · Prioridad: Alta · Sprint 1 · Actor: Visitante
Descripción	Yo como Visitante, quiero poder crear una cuenta, para reseñar y guardar películas. Tomando los datos: nombre de usuario, correo y contraseña.
Funcionalidad	El backend valida los datos y guarda la contraseña con hash. En el frontend, pages/login.html muestra el formulario de registro, avisa si la contraseña tiene menos de 8 caracteres sin enviar nada y, si es válida, inicia sesión sola.
Criterios de aceptación	1. El sistema debe crear la cuenta con rol “user” y no devolver la contraseña. 2. El sistema debe rechazar un correo o un nombre de usuario ya registrado (409). 3. El sistema debe rechazar una contraseña fuera de 8 a 72 caracteres, sin llamar al servidor. 4. Tras registrarse, el usuario debe quedar con la sesión iniciada.
Restricciones	Correo y nombre de usuario son únicos.

HU-02 · Iniciar y cerrar sesión

Código del requerimiento	RF02 (HU-02) · Prioridad: Alta · Sprint 1 · Actor: Usuario
Descripción	Yo como Usuario, quiero poder iniciar sesión, para usar las funciones reservadas a quienes tienen cuenta. Tomando los datos: correo y contraseña.
Funcionalidad	POST /auth/login devuelve un token JWT. El frontend lo guarda en localStorage (clave marisolliflix_sesion), lo envía en cada ruta protegida y sincroniza la sesión entre pestañas. Salir pide confirmación.
Criterios de aceptación	1. El sistema debe devolver un token con credenciales correctas y 401 con credenciales incorrectas. 2. El sistema debe bloquear el login tras 5 intentos fallidos en 15 minutos (429). 3. El menú debe cambiar según la sesión y mostrar “Salir”. 4. El enlace ?volver= debe aceptar solo rutas del propio sitio.
Restricciones	El token expira según JWT_EXPIRES. El backend decide los permisos reales.

HU-03 · Distinguir usuarios y administradores

Código del requerimiento	RF03 (HU-03) · Prioridad: Alta · Sprint 1 · Actor: Administrador
Descripción	Yo como Administrador, quiero poder tener permisos especiales, para moderar el catálogo. Tomando los datos: el rol del usuario.
Funcionalidad	El middleware authorize('admin') protege las rutas de administración y un seed crea al administrador desde las variables ADMIN_* del .env. El frontend oculta las opciones de administrador a los demás.
Criterios de aceptación	1. El sistema debe responder 403 si un usuario común usa una ruta de administrador. 2. El seed debe crear al administrador una sola vez, aunque se ejecute varias veces. 3. El panel de administración debe mostrarse solo a administradores.
Restricciones	El rol viaja dentro del token; se obtiene al iniciar sesión.

HU-04 · Gestionar categorías

Código del requerimiento	RF04 (HU-04) · Prioridad: Alta · Sprint 1 · Actor: Administrador
Descripción	Yo como Administrador, quiero poder crear, editar y eliminar categorías, para organizar el catálogo. Tomando los datos: nombre de la categoría.

Funcionalidad	CRUD en /categories solo para el administrador, con lectura pública. El panel (pestaña Categorías) lo consume y confirma antes de borrar.
Criterios de aceptación	1. El sistema debe rechazar un nombre repetido, sin distinguir mayúsculas (409). 2. El sistema debe rechazar eliminar una categoría con películas (409). 3. Cualquier visitante debe poder consultar las categorías para filtrar el listado.
Restricciones	Solo el administrador crea, edita o elimina.

HU-05 · Registrar películas, series y animes

Código del requerimiento	RF05 (HU-05) · Prioridad: Alta · Sprint 1 · Actor: Usuario / Administrador
Descripción	Yo como Usuario, quiero poder registrar una película, serie o anime, para ampliar el catálogo. Tomando los datos: título, descripción, categoría, año, tipo y, opcionalmente, portada, trailer y plataformas.
Funcionalidad	CRUD de películas con título único. pages/proponer.html envía la propuesta; el administrador edita y elimina desde el panel. El trailer solo acepta enlaces de YouTube y las plataformas salen de una lista cerrada.
Criterios de aceptación	1. El sistema debe rechazar un título repetido, sin distinguir mayúsculas (409). 2. El sistema debe rechazar datos inválidos mostrando los errores del backend. 3. Solo el administrador debe poder editar y eliminar.
Restricciones	El título es único. El administrador que propone publica directo.

HU-06 · Aprobar o rechazar propuestas

Código del requerimiento	RF06 (HU-06) · Prioridad: Alta · Sprint 1 · Actor: Administrador
Descripción	Yo como Administrador, quiero poder aprobar o rechazar las películas propuestas, para mantener el catálogo confiable. Tomando los datos: el estado (approved o rejected).
Funcionalidad	Las propuestas quedan en estado “pending”. PATCH /movies/:id/status las aprueba o rechaza y la pestaña Pendientes del panel lo hace con un clic.
Criterios de aceptación	1. El sistema debe ocultar las películas no aprobadas al público. 2. El sistema debe permitir al administrador ver las pendientes, de las más viejas a las más nuevas. 3. El sistema debe rechazar el estado “pending” como destino (422).
Restricciones	Quien propone puede ver su propia propuesta aunque esté pendiente.

HU-08 · Escribir una reseña con calificación

Código del requerimiento	RF07 (HU-08) · Prioridad: Alta · Sprint 1 · Actor: Usuario
Descripción	Yo como Usuario, quiero poder escribir una reseña de una película, para compartir mi opinión. Tomando los datos: título, comentario y calificación de 1
	a 10.
Funcionalidad	POST /movies/:id/reviews dentro de una transacción. El formulario del detalle valida en el navegador (sin solo espacios y con la calificación elegida) antes de enviar.
Criterios de aceptación	1. El sistema debe aceptar una calificación entera de 1 a 10 y rechazar otras (422). 2. El sistema debe permitir una sola reseña por usuario y película (409). 3. La reseña nueva debe actualizar el ranking de la película.
Restricciones	Requiere sesión. La película debe estar aprobada.

HU-09 · Editar y eliminar mi reseña

Código del requerimiento	RF08 (HU-09) · Prioridad: Normal · Sprint 2 · Actor: Usuario
Descripción	Yo como Usuario, quiero poder editar o eliminar mi reseña, para corregir mi opinión. Tomando los datos: los campos que cambian.
Funcionalidad	PUT y DELETE /reviews/:id en transacción; solo el autor puede usarlos. En la interfaz, la reseña propia muestra los botones Editar y Eliminar con confirmación.
Criterios de aceptación	1. El sistema debe responder 403 si la reseña es de otra persona. 2. Al eliminarla deben eliminarse también sus votos. 3. Al editar debe conservarse la calificación anterior hasta que se cambie.
Restricciones	Se debe enviar al menos un campo al editar.

HU-10 · Dar like o dislike a una reseña

Código del requerimiento	RF09 (HU-10) · Prioridad: Normal · Sprint 2 · Actor: Usuario
Descripción	Yo como Usuario, quiero poder dar like o dislike a las reseñas de otros, para destacar las más útiles. Tomando los datos: el tipo de voto.
Funcionalidad	POST y DELETE /reviews/:id/reaction. Repetir el mismo tipo quita el voto y el otro tipo lo cambia. En la interfaz son botones con aria-pressed; en la reseña propia son texto plano.
Criterios de aceptación	1. El sistema debe impedir votar la propia reseña (403). 2. El sistema debe contar un solo voto por usuario y reseña. 3. El conteo debe actualizarse en pantalla al votar.
Restricciones	Requiere sesión.

HU-11 · Calcular un ranking ponderado

Código del requerimiento	RF10 (HU-11) · Prioridad: Alta · Sprint 2 · Actor: Visitante
Descripción	Yo como Visitante, quiero ver un ranking justo de las películas, para decidir qué ver. Tomando los datos: calificaciones, likes, dislikes y antigüedad de las reseñas.
Funcionalidad	Las vistas review_weights y movie_stats calculan el puntaje: peso = max(0.1, 1 + 0.2 × (likes − dislikes)) ÷ (1 + días/365); puntaje bayesiano con 5.5 y 2 votos de relleno. Desempata por reseñas, promedio, likes, IMDb y título.
Criterios de aceptación	1. El sistema debe dar puntaje 0 a una película sin reseñas. 2. El sistema debe dar menos peso a las reseñas más viejas. 3. El Top 5 no debe cambiar según el límite pedido.
Restricciones	Los datos derivados no se guardan; se calculan en la base.

HU-12 · Explorar el listado con orden, filtro y paginación

Código del requerimiento	RF11 (HU-12) · Prioridad: Alta · Sprint 2 · Actor: Visitante
Descripción	Yo como Visitante, quiero explorar el catálogo, para encontrar qué ver.
	Tomando los datos: categoría, orden, texto de búsqueda y página.
Funcionalidad	GET /movies con sort, categoryId, q, page y limit. pages/listado.html muestra 36 tarjetas por página en una cuadrícula de 6 columnas; el inicio muestra la portada destacada, el Top 5 y los más populares.
Criterios de aceptación	1. El sistema debe usar el orden, el filtro y la búsqueda del backend. 2. El listado debe mostrar los estados de carga, vacío y error. 3. La búsqueda debe escapar los caracteres % y _.
Restricciones	Solo se listan películas aprobadas.

HU-13 · Ver el detalle de una película

Código del requerimiento	RF12 (HU-13) · Prioridad: Normal · Sprint 3 · Actor: Visitante
Descripción	Yo como Visitante, quiero ver el detalle de una película, para conocer su sinopsis y las opiniones. Tomando los datos: el identificador de la película.
Funcionalidad	El detalle abre en una ventana emergente (elemento dialog) al pulsar cualquier tarjeta y también existe como página completa con trailer, "dónde ver", nota de IMDb y reseñas.
Criterios de aceptación	1. La ventana debe cerrarse con Escape y devolver el foco. 2. El sistema debe mostrar "—" en las estadísticas cuando no hay reseñas. 3. Un título no aprobado solo debe verlo el administrador o quien lo propuso (404).
Restricciones	Escribir reseñas y votar requiere sesión.

HU-14 · Administrar el catálogo desde un panel

Código del requerimiento	RF13 (HU-14) · Prioridad: Alta · Sprint 2 · Actor: Administrador
Descripción	Yo como Administrador, quiero un panel de administración, para gestionar categorías, películas y propuestas. Tomando los datos: los de cada categoría o película.
Funcionalidad	pages/admin.html con pestañas Categorías, Películas y Pendientes. Pide sesión de administrador y confirma las acciones destructivas.
Criterios de aceptación	1. El panel debe estar disponible solo para administradores. 2. El administrador debe poder aprobar o rechazar propuestas. 3. Los mensajes de error del backend deben mostrarse en pantalla.
Restricciones	El backend devuelve 401 o 403 si no corresponde.

HU-15 · Responder con errores claros y seguros

Código del requerimiento	RF15 (HU-15) · Prioridad: Alta · Sprint 1 · Actor: Todos
Descripción	Yo como Usuario, quiero recibir mensajes claros cuando algo falla, para saber qué corregir. Tomando los datos: cualquier petición al API.
Funcionalidad	Manejador central con formato { status, message, errors[] }, validación con express-validator, límite de peticiones, CORS restringido y consultas parametrizadas.
Criterios de aceptación	1. El sistema debe responder con el código HTTP correcto (400, 401, 403, 404, 409, 413, 422, 429 y 500). 2. El error 500 no debe revelar detalles internos. 3. Una petición con JSON mal formado debe responder 400.
Restricciones	Todos los errores salen por el mismo manejador.

HU-16 · Documentar el API con Swagger

Código del requerimiento	RF16 (HU-16) · Prioridad: Normal · Sprint 2 · Actor: Desarrollador
Descripción	Yo como Desarrollador, quiero consultar la documentación del API, para
	integrar el frontend. Tomando los datos: la especificación OpenAPI.
Funcionalidad	Swagger UI en /api/v1/docs y la especificación en /api/v1/docs.json. La versión del API sigue semver (1.0.0).
Criterios de aceptación	1. La documentación debe listar todos los endpoints con ejemplos. 2. Debe poder probarse un endpoint protegido con Authorize y el token.
Restricciones	La documentación debe coincidir con las rutas reales.

Metodología

El proyecto se desarrolló con la metodología ágil Scrum, adaptada a un equipo pequeño. El trabajo se dividió en tres sprints cortos, cada uno con un objetivo claro y un conjunto de historias de usuario priorizadas en el tablero de ClickUp. Las fechas de los sprints corresponden a la actividad registrada en el historial de Git de los dos repositorios.

Roles

Tabla 3

Roles del equipo Scrum

Rol	Responsable	Responsabilidades
Product Owner	María José Montepeque Zet	Definir y priorizar las historias de usuario a partir del enunciado y mantener el backlog en ClickUp.
Scrum Master	María José Montepeque Zet	Organizar los sprints, dar seguimiento al tablero, revisar los Pull Requests y resolver los bloqueos.
Developer	María José Montepeque Zet	Backend (categorías, películas, aprobación, ranking, listado, Swagger, seguridad, POO y seeds), pantallas del frontend, despliegue y documentación.
Developer	Allison Rocío Vargas Mejía	Autenticación y registro (HU-01), roles y seeder del administrador (HU-03), reseñas (HU-08), y búsqueda, trailer, “dónde ver”, IMDb y Mi lista.
Stakeholder	Eduin Salas	Entregar los requisitos del proyecto y evaluar el producto final.

Sprints

Tabla 4

Sprints del proyecto

Sprint	Fechas	Objetivo	Historias de usuario
Sprint 1 · Fundación	26 y 27 de octubre de 2026	Construir la base del sistema: estructura de ambos repositorios, base de datos MySQL, autenticación con JWT, roles, categorías, películas con aprobación, primera versión de las reseñas y manejo de errores.	HU-01 a HU-06, HU-08 y HU-15 (8 HU)
Sprint 2 · Reseñas, ranking y pantallas	28 y 29 de octubre de 2026	Completar el ciclo de las reseñas (editar, eliminar y votar), el ranking ponderado y el orden del listado; conectar las pantallas del frontend con la API, crear el panel de administración, documentar con Swagger y desplegar en el servidor.	HU-09 a HU-12, HU-14 y HU-16 (6 HU)
Sprint 3 · Catálogo, mejoras y entrega	30 y 31 de octubre de 2026	Detalle de película con ventana emergente, catálogo de 112 títulos con portada, arquitectura POO, mejoras de accesibilidad, rendimiento y seguridad (informe de 19 puntos), búsqueda, trailer, “dónde ver”, IMDb y Mi lista; documentación final y entrega.	HU-13 y mejoras (ver Tabla 5)

Eventos de Scrum

- Sprint Planning: al inicio de cada sprint se seleccionaron las historias del backlog y se dividieron en subtareas dentro de ClickUp.
- Daily Stand Up: revisión diaria de lo avanzado, lo pendiente y los bloqueos, actualizando el estado de las tarjetas.
- Sprint Review: al cerrar cada sprint se probó el sistema contra los criterios de aceptación de las historias, con la lista de comprobación manual del README y las pruebas automáticas del backend.
- Sprint Retrospective: al final se analizó qué salió bien, qué costó y qué se puede mejorar (ver la sección de conclusiones).

Flujo de trabajo en Git

Se usó un flujo basado en Git Flow: la rama main contiene solo la versión de entrega y su merge despliega automáticamente, la rama dev integra las funcionalidades terminadas y cada historia o tarea se trabaja en su propia rama que sale de dev (feature/hu-XX-descripcion, fix/, docs/, chore/ y refactor/). Cada rama se integra a dev mediante un Pull Request con la plantilla del repositorio, que exige el checklist de tecnología, consumo de la API, roles, interfaz y calidad, y los commits siguen el formato Conventional Commits (feat, fix, refactor, docs, chore, test).

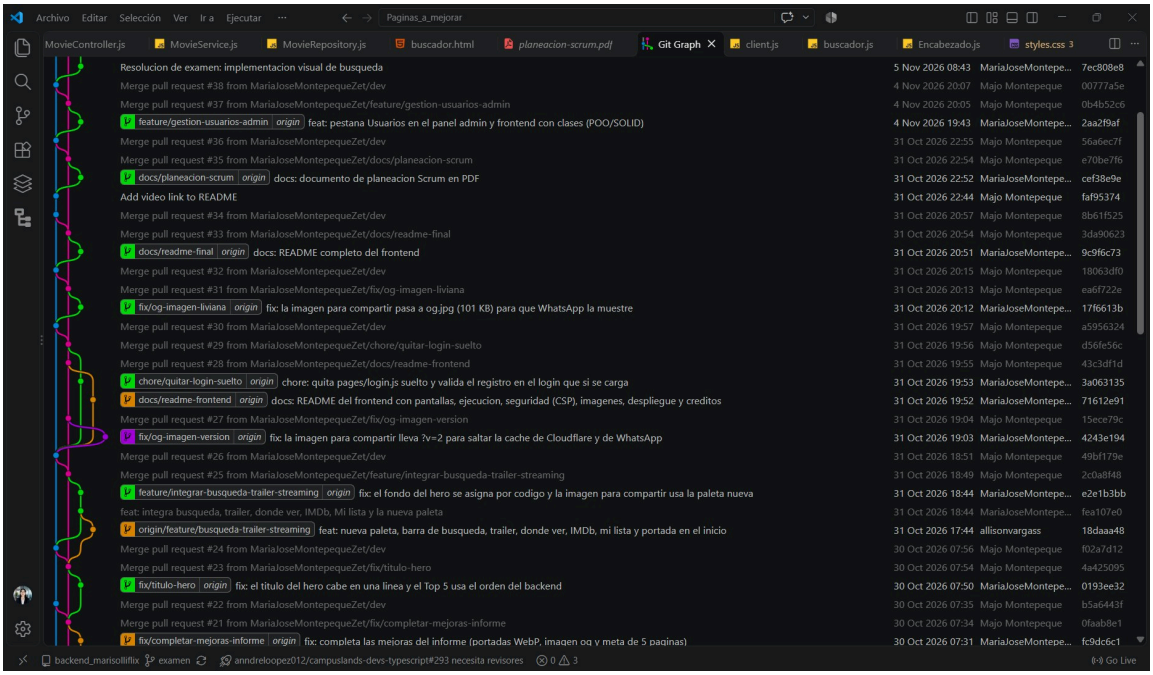
Evidencia de planteamiento de plataforma de trabajo

Repositorios

- Frontend: https://github.com/MariaJoseMontepequeZet/frontend_marisolliflix (64 commits y 32 Pull Requests integrados).
- Backend: https://github.com/MariaJoseMontepequeZet/backend_marisolliflix (103 commits y 54 Pull Requests integrados).
- Sitio publicado: https://marisolliflix.pamcing.me · Documentación Swagger: https://marisolliflix.pamcing.me/api/v1/docs
- Video de la presentación: https://drive.google.com/drive/folders/1fWOdx8Gpee_4csEE-WjLwmUCvh51XfC4?usp=drive_link
- Ramas del frontend: main, dev, feature/estructura, feature/hu-12-pantallas-inicio-listado, feature/hu-13-detalle-pelicula, feature/portadas-peliculas, feature/ventana-emergente-pelicula, feature/integrar-busqueda-trailer-streaming, fix/completar-mejoras-informe, docs/readme-final, entre otras.
- Ramas del backend: main, dev, feature/hu-01-hu-02-auth-mysql, feature/hu-03-roles-admin, feature/hu-04-categorias, feature/hu-05-peliculas, feature/hu-06-aprobacion-peliculas, feature/hu-08-create-review, feature/hu-09-editar-eliminar-resenas, feature/hu-10-like-dislike-resenas, feature/hu-11-ranking-ponderado, feature/hu-12-listado-orden, feature/hu-15-seguridad-errores, feature/hu-16-swagger, entre otras.
- Ejemplos de commits: “feat(hu-11): ranking ponderado por calificacion, likes/dislikes y fecha de resena”, “[HU-13][FE] Pantalla de detalle de pelicula con resenas, likes y dislikes”, “refactor: repositorios con herencia y servicios con clases (POO)”, “fix: el ranking desempata por resenas, promedio, likes y titulo”, “ci: desplegar automaticamente en el servidor al hacer push a main”.

Figura 1

Historial de ramas y commits de los repositorios en GitHub



Tablero Scrum (ClickUp)

El tablero se organizó con una lista por sprint. Cada historia de usuario tiene responsable, fecha límite, prioridad y subtareas.

Figura 2

Vista general del espacio de MarisolliFlix en ClickUp

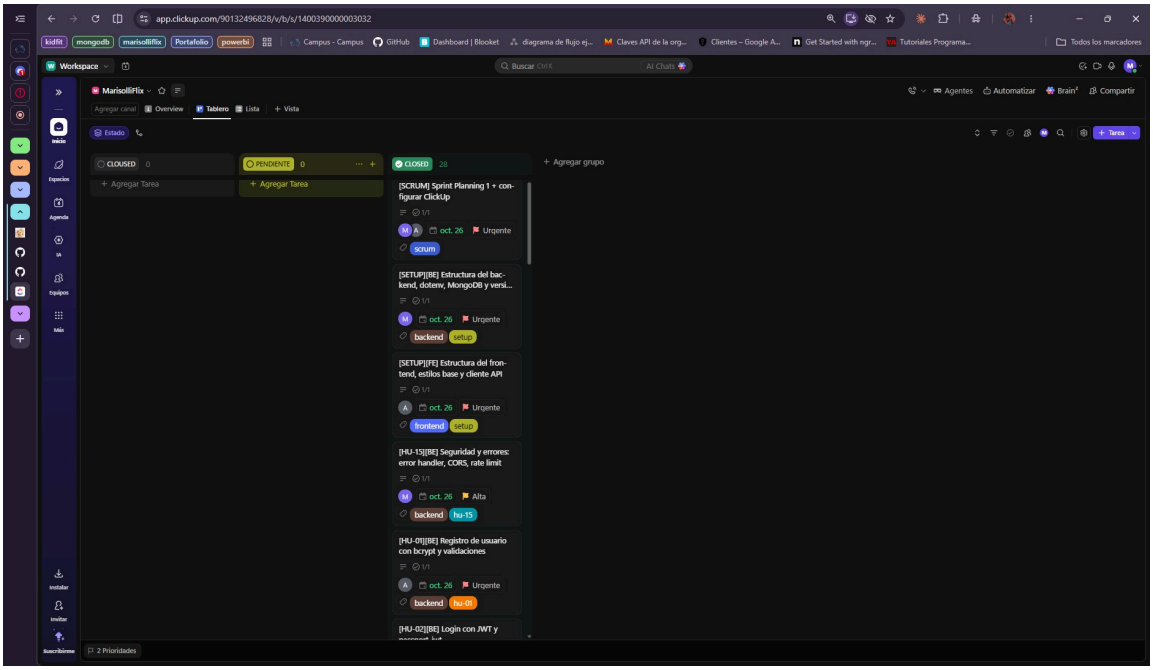
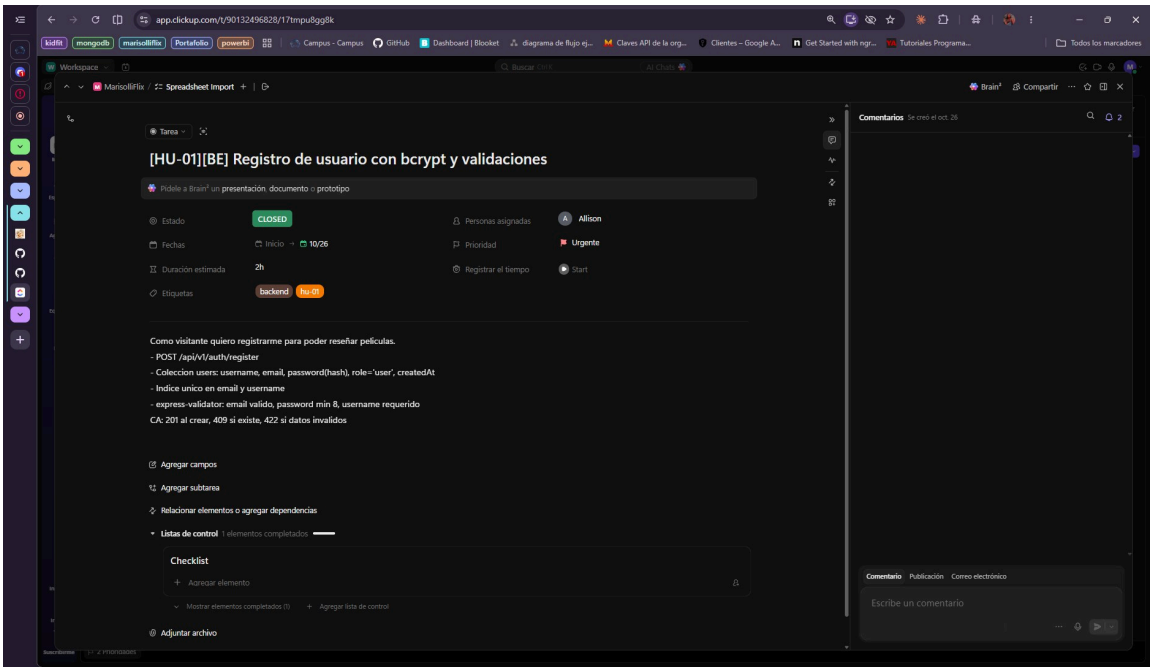


Figura 3

Historia de usuario HU-01 registrada en ClickUp



Resultados y funcionamiento del producto final

Tabla 5

Mejoras incorporadas en el Sprint 3 además de las historias de usuario

Mejora	Qué se hizo
Catálogo de 112 títulos	Seed con sinopsis (Wikipedia en español, CC BY-SA) y portada de cada película, sustituyendo el catálogo anterior de 400.
Portadas livianas	Cada portada en JPG, WebP de 400 px y WebP de 220 px con srcset (−75 % de peso en las tarjetas).
Arquitectura POO	DTOs, entidades, repositorios con herencia y servicios con inyección de dependencias en el backend.
Informe de 19 puntos	Top 5 con ranking real, sesión sincronizada entre pestañas, :focus-visible, prefers-reduced-motion, textos legibles, menú hamburguesa, pie de página y Open Graph.
Búsqueda, trailer y dónde ver	Buscador en el encabezado, trailer con youtube-nocookie.com y plataformas de streaming por película.
Nota de IMDb y Mi lista	Calificación pública como referencia, orden “Internet” y lista personal de películas guardadas.
Imagen para compartir	og.jpg de 101 KB (WhatsApp descarta las de más de 300 KB) con versión en la URL para saltar la caché.
Seguridad del sitio	Sin estilos ni scripts en línea para respetar la CSP estricta, y texto externo escapado antes de mostrarlo.

Tabla 6

Tecnologías utilizadas

Tecnología	Uso en el proyecto
HTML, CSS y JavaScript puro (módulos ES)	Interfaz web sin frameworks ni proceso de compilación.
Node.js 20+ y Express 5	Servidor de la API REST.
MySQL 8 + mysql2	Base de datos relacional (4FN, 13 tablas y 2 vistas) y driver sin ORM.
passport, passport-jwt, jsonwebtoken y bcryptjs	Autenticación con tokens y contraseñas con hash.
express-validator y express-rate-limit	Validación de entradas y límite de peticiones.
swagger-ui-express	Documentación OpenAPI 3 en /api/v1/docs.
node:test	150 pruebas automáticas del backend.
Docker, Caddy y Cloudflare	Despliegue, servidor web con cabeceras de seguridad, dominio, HTTPS y caché.
GitHub Actions	Despliegue automático con un runner propio al hacer push a main.
ClickUp / GitHub	Seguimiento de Scrum y control de versiones.

Tabla 7

Patrones y conceptos de diseño aplicados

Patrón o concepto	Dónde	Para qué
Repository	src/repositories/ (Repository, RepositoryConId y sus hijas)	Concentra todo el SQL; los servicios nunca escriben consultas.
DTO	src/dtos/	Se queda solo con los campos que le tocan de la petición y define qué se responde.
Entidades	src/models/entities/ (Movie, Category, Review, User)	Representan una fila de MySQL como objeto con atributos privados.
Inyección de	Constructores de los servicios	Permite probar con repositorios falsos sin

Patrón o concepto	Dónde	Para qué
dependencias		tocar la base de datos.
Transacción	conTransaccion en ReviewService	Reseña, votos y ranking quedan consistentes: commit si todo sale bien y rollback si algo falla.
Módulo de cliente de API	js/api/client.js (frontend)	Único lugar que llama a fetch, agrega el token y traduce los errores.

Tabla 8

Principios SOLID aplicados

Principio	Aplicación
S · Responsabilidad única	Cada capa tiene una tarea: las rutas validan, los controladores responden, los servicios aplican reglas de negocio y los repositorios consultan.
O · Abierto/cerrado	Repository se extiende con clases hijas (CategoryRepository, MovieRepository…) sin modificar la clase base.
L · Sustitución de Liskov	Cualquier repositorio hijo se puede usar donde se espera un Repository, porque todos implementan hidratar(fila).
I · Segregación de interfaces	ReviewRepository y WatchlistRepository, de tablas sin id propio, heredan directo de Repository y no prometen findById.
D · Inversión de dependencias	Cada servicio recibe su repositorio por el constructor en lugar de crearlo.

Consistencia de datos: crear, editar y eliminar reseñas y registrar o cambiar likes y dislikes se ejecutan con conTransaccion, que hace commit si todo sale bien y rollback si algo falla. El resultado se verifica con las pruebas automáticas de reseñas, reacciones y ranking.

Figura 4

Diagrama entidad-relación de la base de datos marisolliflix

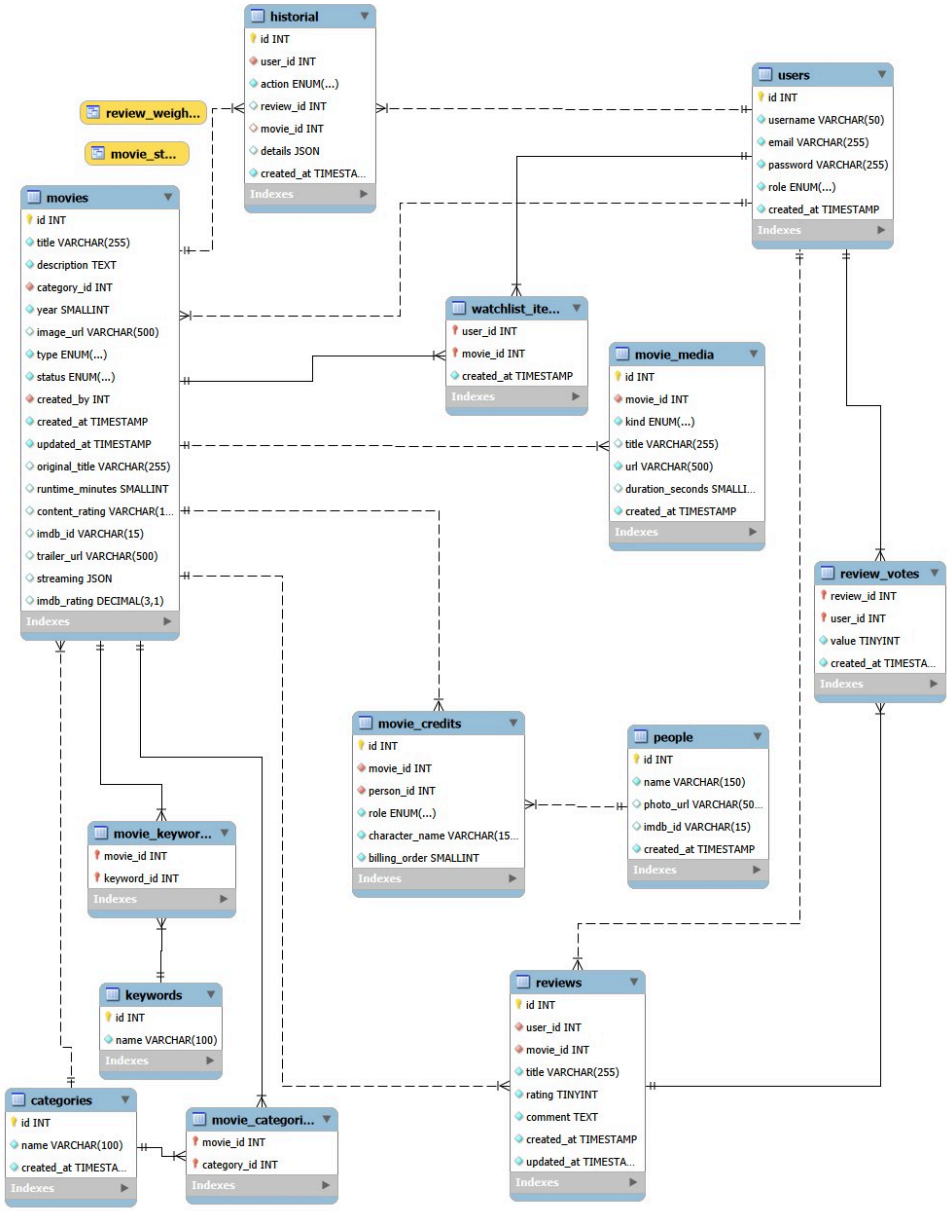


Figura 5

Pantalla de inicio con la película destacada, el Top 5 y los más populares

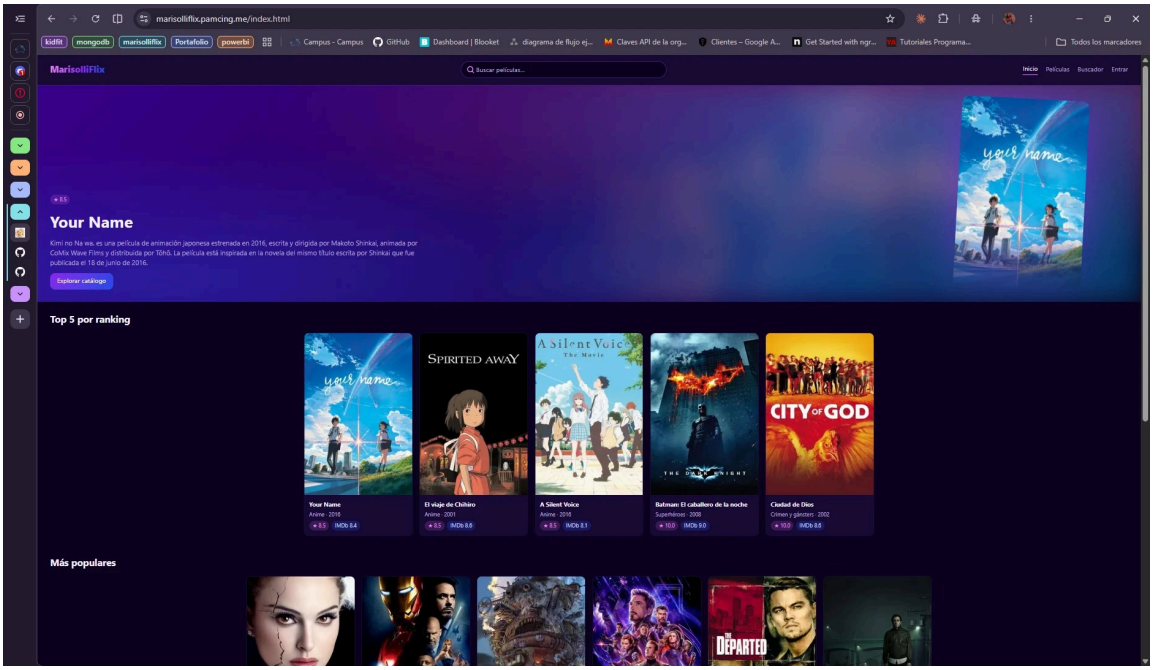


Figura 6

Detalle de una película con trailer, dónde ver y reseñas

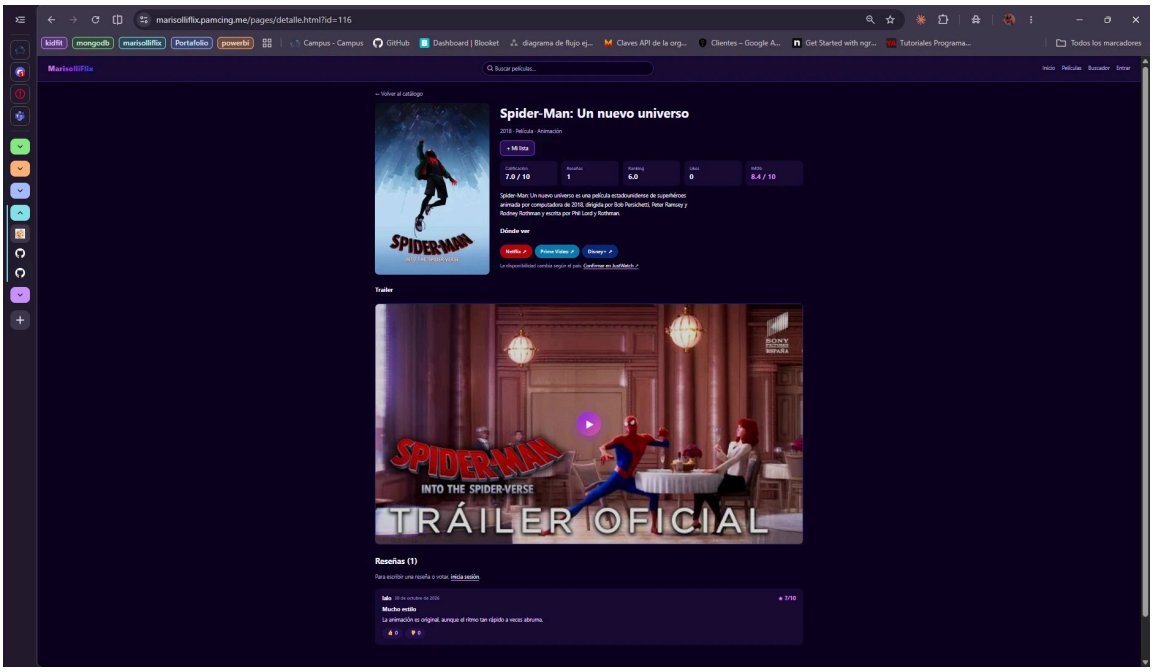
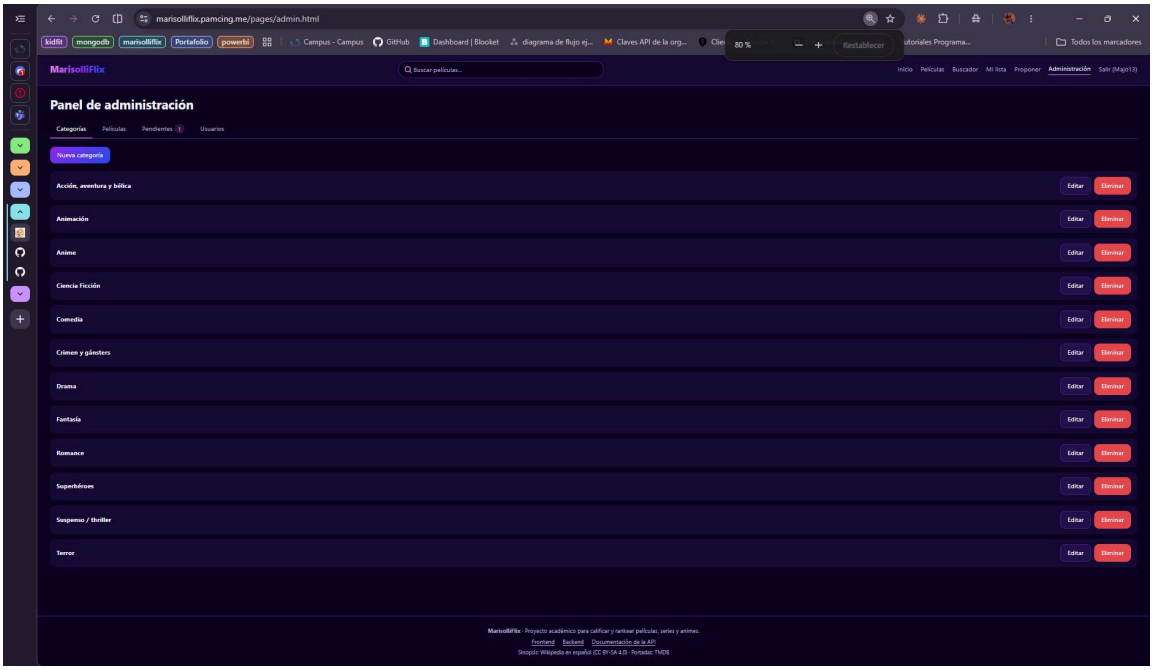


Figura 7

Panel de administración



Conclusiones

Conclusiones generales

- Se desarrolló una aplicación web con frontend y backend independientes que cubre el registro y la calificación de películas, series y animes, con reseñas, votos, ranking ponderado, Mi lista y moderación del catálogo, cumpliendo las historias de usuario planteadas.
- La arquitectura por capas y los principios SOLID permitieron avanzar historia por historia sin romper lo que ya funcionaba, y la refactorización a clases (POO) se pudo hacer sobre el sistema ya funcionando gracias a las pruebas.
- Calcular el ranking en vistas SQL, en lugar de guardar totales en la tabla de películas, evita que los datos queden desincronizados.
- Las transacciones garantizan que reseñas y votos se apliquen completos o no se apliquen.
- Scrum, ClickUp y el flujo de ramas con Pull Requests ayudaron a dividir un proyecto grande en entregas pequeñas dentro de un plazo corto, y el despliegue automático permitió ver cada entrega en el sitio real.

Conclusiones de la retrospectiva del sprint

Qué salió bien

- Trabajar cada tarea en su propia rama con Pull Request hacia dev mantuvo ordenados los dos repositorios.
- Construir primero la base (autenticación, roles y películas) hizo que reseñas, ranking y pantallas avanzaran más rápido.
- Las 150 pruebas automáticas dieron seguridad al refactorizar a POO y al integrar el trabajo de varias personas.
- El despliegue automático al hacer push a main acortó el ciclo entre terminar una tarea y verla publicada.

Qué costó

- El catálogo cambió de 400 a 112 títulos y hubo que escribir un script para quitar los sobrantes sin borrar propuestas de usuarios.
- La política de seguridad del sitio (CSP) no permite estilos ni scripts en línea, así que varias pantallas se tuvieron que ajustar.
- La imagen para compartir pesaba demasiado para WhatsApp y la caché de Cloudflare y de las aplicaciones de mensajería mostraba la versión vieja.

- El enunciado pedía MongoDB y el equipo usa MySQL; hubo que documentar la decisión y adaptar las transacciones y el ranking.
- Algunas secciones de los README quedaron con marcas “[completar]” hasta el final.

Qué mejorar

- Definir el modelo de datos completo antes de redactar las tareas técnicas del tablero.
- Actualizar el estado de las tarjetas en ClickUp en el momento en que se termina cada tarea.
- Guardar el token fuera de localStorage (por ejemplo en una cookie HttpOnly) y aprovechar las tablas de reparto y palabras clave que el API todavía no usa.
- Completar los roles del equipo en los README y revisar el contenido de cada archivo antes de cada commit.